

A Scalable Social Media Meme Recommendation System Using Graph-Based Network and Database Indexing

1 Work till now

The main database optimization required in our project revolves around fast graph queries with the help of a relational database like postgres. Some checkpoints covered till now are:

- Benchmarking various indexes with respect to graph algorithms like DFS, BFS, K-hop to find the most optimal one for our use case.
- Simulating on different graph sizes to approximate the expected time complexity of various operations that would be used in our application(k-hop neighbour return, adjacency list, membership check)
- Forming a rough system design for our application, though subject to change

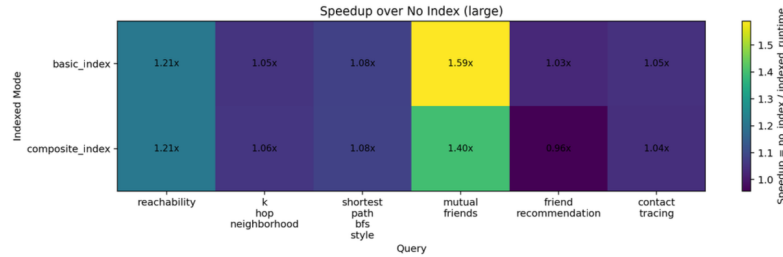


Figure 6: Speedup heatmap on the large dataset.

2 System Overview

The application will consist of two major components: social graph management, and recommendation generation. A user requests a feed. The backend then identifies a set of relevant accounts using a k-hop traversal over the connection graph, extracts photos liked by those accounts, filters out photos already viewed in recent days, and finally adds a small number of random photos to keep the feed diverse.

The system uses the following relations in PostgreSQL:

- **account_viewed_photo**: stores which photos an account has viewed and when.
- **account_liked_photo**: stores which photos an account has liked and when.
- **account_account**: stores user-to-user connections in the social graph.

These relations support both retrieval and analytics. The recommendation pipeline is expected to issue repeated lookups on account identities, graph neighbors, and recent photo history, which makes indexing an important optimization target.

3 Recommendation Logic

For a given user u , the feed generation process is as follows:

1. Find the set of accounts within k hops from u in the connection graph.
2. Collect photos liked by those accounts, giving higher weight to photos liked by closer accounts.
3. Remove photos that u has viewed in the recent time window.
4. Add a controlled fraction of random photos to improve exploration and prevent overfitting to the same content.
5. Rank the remaining candidates using a scoring rule based on hop distance, number of likes among neighbors, recency, and diversity.

4 Database Design and Indexing

Because PostgreSQL is relational, naive queries over very large tables can become expensive. The feed query touches three common access patterns: finding connections, retrieving liked photos, and checking recent views. For this reason, indexing strategy is a central part of the project.

Likely useful indexes include:

- B-tree indexes on account identifiers in all three relations.
- Composite indexes such as $(account_id, photo_id)$ for fast membership checks.
- Indexes on timestamp columns to efficiently filter recent views and likes.
- Indexes on the connection table to accelerate graph traversal from a source account.

We will compare alternative index configurations through simulation.

5 Simulation Plan

We will generate synthetic data with configurable numbers of users, photos, connections, likes, and views. The simulation will run the feed-generation query under different settings:

- no index baseline,
- single-column indexes,
- composite indexes,
- timestamp-based filtering indexes,
- combined multi-index strategy.

The results will show which indexing approach gives the best trade-off between speed and storage overhead.